

Git + GitHub

SSH Connection Guide

PW EARNERS : Coding and Web Development (Lecture 28)

By : Nishant Saini Sir

A beginner-friendly setup guide for macOS and Windows

From installing Git → creating a GitHub account → generating SSH keys → connecting GitHub → cloning, committing and pushing

1. What You Are Actually Setting Up

Before running commands, understand the three things involved:

Git	GitHub	SSH
Git	GitHub	SSH

2. The Big Picture

The connection you are building looks like this:

Your Computer → Git → SSH Key → GitHub Account → GitHub Repository

There are two SSH keys. They are created together, but they do different jobs:

- Private key: stays on your computer. NEVER upload or send it to anyone.
- Public key: can be uploaded to GitHub. This is the key you copy into GitHub.

Think of the private key as your secret physical key and the public key as the lock information GitHub uses to recognize that key.

3. What You Need Before Starting

- A GitHub account.
- Git installed on your computer.
- A terminal application: Terminal on macOS or Git Bash/PowerShell on Windows.
- An internet connection.
- The email address associated with your GitHub account is useful when creating the SSH key.

4. Install Git

macOS

Open Terminal and run:

```
git --version
```

If Git is available, you will see a version such as git version 2.x.x. If macOS asks you to install Command Line Developer Tools, accept the installation and wait for it to finish.

Windows

Install Git for Windows. During installation, the default options are normally suitable for beginners. After installation, open Git Bash and run:

```
git --version
```

If a Git version appears, Git is installed correctly.

5. Configure Your Git Name and Email

This step tells Git which name and email should be attached to the commits you create.

Run these commands on both macOS and Windows:

```
git config --global user.name "Your Name"
git config --global user.email "your-email@example.com"
```

Check what Git saved:

```
git config --global --list
```

Important: this is Git's commit identity. It is separate from SSH authentication.

6. Create Your GitHub Account

If you do not already have a GitHub account, create one on GitHub. Use an email address you can access. After signing in, you will manage your repositories from your GitHub account.

7. SSH Setup - What We Will Do

1. Check whether an SSH key already exists.
2. Create an Ed25519 SSH key if needed.
3. Start/use the SSH agent.
4. Add the private key to the SSH agent.
5. Copy the public key.
6. Add the public key to GitHub.
7. Test the connection.
8. Use the SSH repository URL when cloning or changing a remote.

PART A - macOS SSH SETUP

8. Open Terminal

Press Command + Space, type Terminal, and open it.

You can also find Terminal in Applications → Utilities → Terminal.

9. Check for an Existing SSH Key

```
ls -al ~/.ssh
```

Look for files such as id_ed25519 and id_ed25519.pub. The .pub file is the public key.

If you already have an SSH key that you use for another service, do not automatically overwrite it. The safest beginner approach is to keep an existing key and create a separately named key if necessary.

10. Generate a New SSH Key

```
ssh-keygen -t ed25519 -C "your-email@example.com"
```

You will see a prompt similar to:

```
Enter file in which to save the key (/Users/YourName/.ssh/id_ed25519):
```

If you are setting up your first key, press Enter to accept the default location.

Next you will be asked for a passphrase.

```
Enter passphrase (empty for no passphrase):
```

A passphrase adds protection to your private key. You can enter one and then enter it again when asked. If you leave it empty, press Enter twice.

11. Start the SSH Agent on macOS

```
eval "$(ssh-agent -s)"
```

Then add your private key:

```
ssh-add --apple-use-keychain ~/.ssh/id_ed25519
```

If you created the key without a passphrase, use:

```
ssh-add ~/.ssh/id_ed25519
```

12. Copy the Public Key on macOS

```
pbcopy < ~/.ssh/id_ed25519.pub
```

The public key is now copied to your clipboard. Do not copy the private key (id_ed25519 without .pub).

13. Add the Public Key to GitHub

9. Sign in to GitHub.
10. Click your profile picture in the top-right corner.
11. Open Settings.
12. Open SSH and GPG keys.

13. Click the New SSH key.
14. Enter a useful title, for example: MacBook Pro.
15. Keep Key type as Authentication Key.
16. Paste the public key into the Key field.
17. Click Add SSH key.

GitHub requires the public key to be added to your account before the SSH connection can authenticate.

14. Test the macOS SSH Connection

```
ssh -T git@github.com
```

The first time, you may be asked whether you want to continue connecting. Verify the GitHub host fingerprint if prompted, then type yes.

A successful authentication message will identify your GitHub username and say that GitHub does not provide shell access. That message means the SSH authentication worked.

PART B - WINDOWS SSH SETUP

15. Open Git Bash

After installing Git for Windows, open the Start menu and search for Git Bash. Open Git Bash.

For the commands in this section, Git Bash is recommended because its commands closely match the macOS/Linux commands.

16. Check for an Existing SSH Key

```
ls -al ~/.ssh
```

Look for id_ed25519 and id_ed25519.pub. If they already exist, do not overwrite them without checking whether they are already being used.

17. Generate a New SSH Key

```
ssh-keygen -t ed25519 -C "your-email@example.com"
```

When asked where to save the key, press Enter for the default location if this is your first key.

```
Enter file in which to save the key (/c/Users/YourName/.ssh/id_ed25519):
```

Choose a passphrase when prompted, or press Enter twice to leave it empty.

18. Enable the Windows SSH Agent

Open **PowerShell as Administrator**. Search for PowerShell, right-click it, and choose Run as administrator.

```
Get-Service -Name ssh-agent | Set-Service -StartupType Manual  
Start-Service ssh-agent
```

Close the administrator PowerShell window. Open Git Bash normally (not as Administrator).

19. Add the Private Key to the SSH Agent

```
ssh-add ~/.ssh/id_ed25519
```

If the command succeeds, the private key has been added to the agent. Remember: you do not upload this private key to GitHub.

20. Copy the Public Key on Windows

```
cat ~/.ssh/id_ed25519.pub | clip
```

The public key is now copied to your clipboard.

21. Add the Public Key to GitHub

18. Sign in to GitHub.
19. Click your profile picture → Settings.
20. Open SSH and GPG keys.
21. Click the New SSH key.
22. Give the key a recognizable title, such as Windows Laptop.
23. Select Authentication Key.
24. Paste the copied public key.

25. Click Add SSH key.

22. Test the Windows SSH Connection

```
ssh -T git@github.com
```

If GitHub asks whether you want to continue connecting, type yes after verifying the host fingerprint. A successful response will contain your GitHub username and indicate that shell access is not provided.

PART C - CONNECT A PROJECT TO GITHUB

23. Create a New Repository on GitHub

26. Open GitHub.
27. Click the + button in the top-right corner.
28. Select New repository.
29. Enter a repository name, for example: my-first-project.
30. Choose Public or Private.
31. If you already have a local project, avoid adding a README, .gitignore, or license during this first setup unless you specifically need them.
32. Click Create repository.

24. Get the SSH Repository URL

Open your repository on GitHub and click the Code button. Select SSH. You will see an address similar to:

```
git@github.com:YOUR-USERNAME/YOUR-REPOSITORY.git
```

Copy that SSH URL. This is the URL you should use when you want Git to communicate with GitHub over SSH.

25. Connect an Existing Local Project

Open Terminal/Git Bash and move into your project folder:

```
cd path/to/your-project
```

Initialize Git if the project is not already a Git repository:

```
git init
```

Check the status:

```
git status
```

Add your files:

```
git add .
```

Create your first commit:

```
git commit -m "Initial commit"
```

Connect the local repository to GitHub:

```
git remote add origin git@github.com:YOUR-USERNAME/YOUR-REPOSITORY.git
```

Verify the remote:

```
git remote -v
```

Finally, push your branch. If your branch is named main:

```
git branch -M main  
git push -u origin main
```

After the push completes, refresh the GitHub repository page. Your project files should appear there.

26. The Normal Daily Workflow

Once the connection is configured, the usual workflow is simple:

```
git status
git add .
git commit -m "Describe what you changed"
git push
```

When you start working on an existing project, first bring down the latest changes when appropriate:

```
git pull
```

A simple mental model:

```
WORK → git add → git commit → git push → GITHUB
```

27. Clone an Existing GitHub Repository

If the project already exists on GitHub and you want a copy on your computer, do not use `git init`. Clone it.

```
git clone git@github.com:YOUR-USERNAME/YOUR-REPOSITORY.git
```

Then enter the project folder:

```
cd YOUR-REPOSITORY
```

Check the remote:

```
git remote -v
```

PART D - UNDERSTANDING WHAT HAPPENS

28. What Does `git init` Do?

`git init` turns your current folder into a Git repository. Git creates a hidden `.git` folder that stores the repository's history and configuration.

```
my-project/  
├── index.html  
├── style.css  
└── .git/
```

29. What Does `git add` Do?

`git add` prepares changes for the next commit. It does not upload anything to GitHub.

```
git add .
```

30. What Does `git commit` Do?

A commit records a snapshot of the changes in your local Git repository. A commit is still only on your computer until you push it.

```
git commit -m "Add homepage"
```

31. What Does `git push` Do?

`git push` sends your local commits to the remote repository on GitHub.

```
git push
```

32. What Does `git pull` Do?

`git pull` retrieves changes from the remote repository and integrates them into your local branch.

```
git pull
```

33. What Is origin?

origin is simply the conventional name Git gives to the remote repository you connected to. It is not a special GitHub account or command.

```
git remote -v
```

34. What Is SSH Actually Doing?

When you run `git push`, Git contacts GitHub using the SSH URL. GitHub asks your computer to prove it has the private key corresponding to a public key registered to a GitHub account. Your private key remains on your computer; GitHub stores the public key.

35. Public Key vs Private Key

File	Where it stays	Can you share it?
<code>id_ed25519</code>	Your computer	NO - keep it secret
<code>id_ed25519.pub</code>	Your computer + GitHub	YES - this is the public key

PART E - TROUBLESHOOTING

36. Error: Permission denied (publickey)

This usually means GitHub could not authenticate your SSH key. Check these items in order:

- Did you generate an SSH key?
- Did you add the private key to the SSH agent?
- Did you add the PUBLIC key (.pub) to the correct GitHub account?
- Are you using the correct GitHub account?
- Are you using an SSH repository URL beginning with git@github.com:?
- Does ssh -T git@github.com succeed?

```
ssh -T git@github.com
```

If this command fails, fix SSH authentication before troubleshooting Git push/pull.

37. Error: Repository not found

Check the remote URL and make sure the repository exists and your GitHub account has access to it.

```
git remote -v
```

For SSH, it should look like:

```
git@github.com:USERNAME/REPOSITORY.git
```

38. Git Push Asks for a Username or Password

Check whether your remote is actually using HTTPS instead of SSH.

```
git remote -v
```

If you see an https://github.com/... URL, change it to the SSH URL:

```
git remote set-url origin git@github.com:USERNAME/REPOSITORY.git
```

```
git remote -v
```

39. Error: remote origin already exists

The project already has a remote named origin. Inspect it first:

```
git remote -v
```

If you need to replace it:

```
git remote set-url origin git@github.com:USERNAME/REPOSITORY.git
```

40. I Accidentally Added the Private Key to Git

Stop. Do not push it. A private SSH key must never be committed or uploaded. If a private key has already been exposed, treat it as compromised and replace/revoke the affected credentials as appropriate.

41. Windows: SSH Agent Problems

Windows can have both the native OpenSSH client and Git for Windows's bundled SSH client. If the key is loaded into one agent but Git is using another SSH client, authentication may still fail. First verify which SSH command Git is using and that the key is loaded.

```
ssh-add -l
```

If you deliberately need Git to use Windows' system OpenSSH client, GitHub documents this configuration:

```
git config --global core.sshCommand "C:/Windows/System32/OpenSSH/ssh.exe"
```

42. SSH Works but git push Still Fails

If `ssh -T git@github.com` successfully authenticates but `git push` fails, SSH itself is probably working. Check the repository URL, branch name, repository permissions, and whether the remote repository has changes that need to be pulled first.

```
git remote -v
git branch --show-current
git status
```

43. Firewall or Network Blocks SSH

Some networks block normal SSH connections. GitHub supports SSH over port 443 as an alternative. This is an advanced fallback; use it only when normal SSH is blocked.

```
ssh -T -p 443 git@ssh.github.com
```

PART F - QUICK CHECKLIST

44. macOS Checklist

- Git installed: `git --version`
- Git name configured
- Git email configured
- Existing SSH keys checked
- Ed25519 SSH key generated
- `ssh-agent` started
- Private key added to `ssh-agent`
- Public key copied
- Public key added to GitHub → Settings → SSH and GPG keys
- `ssh -T git@github.com` succeeds
- Repository remote uses `git@github.com:...`
- `git push` works

45. Windows Checklist

- Git for Windows installed
- Git Bash opened
- Git name configured
- Git email configured
- Existing SSH keys checked
- Ed25519 SSH key generated
- Windows `ssh-agent` service enabled/started
- Private key added with `ssh-add`
- Public key copied
- Public key added to GitHub → Settings → SSH and GPG keys
- `ssh -T git@github.com` succeeds
- Repository remote uses `git@github.com:...`
- `git push` works

46. The Five Commands Beginners Should Remember

```
git status
git add .
git commit -m "your message"
git push
git pull
```

These commands are not the whole Git system, but they cover the basic edit → save → upload → update cycle for many beginner projects.

47. Important Security Rules

- Never send your private SSH key to another person.
- Never upload `id_ed25519` (without `.pub`) to GitHub or any repository.

- The .pub file is the public key; that is the one you add to GitHub.
- Use a different SSH key for different machines when practical, so one lost machine does not require replacing every key.
- If a private key is exposed, treat it as compromised and replace it.
- Do not paste passwords, tokens, or private keys into public issues, repositories, chats, or screenshots.

48. Final Mental Model

1. Git is installed on my computer.
2. Git tracks my project.
3. GitHub stores a remote copy of my Git repository.
4. SSH gives my computer a secure way to authenticate with GitHub.
5. The private key stays on my computer.
6. The public key is added to GitHub.
7. `git remote` connects my local repository to the GitHub repository.
8. `git push` uploads my commits.
9. `git pull` downloads and integrates remote changes.

Once these pieces make sense, Git and GitHub stop feeling like separate systems. Git manages the repository and its history; GitHub hosts the remote repository; SSH authenticates your computer with GitHub.

Official GitHub Documentation

This guide follows GitHub's current SSH setup flow, including Ed25519 keys, ssh-agent configuration, adding the public key to GitHub, and testing the SSH connection.

Recommended references:

- [GitHub Docs - About SSH](#)
- [GitHub Docs - Generating a new SSH key and adding it to the ssh-agent](#)
- [GitHub Docs - Adding a new SSH key to your GitHub account](#)
- [GitHub Docs - Testing your SSH connection](#)